

QUESTION 18

Build a Vector Database Using FAISS and Perform Similarity-Based Document Retrieval

Core Technology: FAISS (Facebook AI Similarity Search)

Embedding Model: all-MiniLM-L6-v2 | Index: IndexFlatIP | Top-K: 5

1. Aim

To build a vector database using FAISS, store document embeddings in a searchable vector index, and retrieve the most similar documents for a user query.

2. Problem Statement

Searching a large collection of documents by comparing every text manually is inefficient. A vector database provides an efficient way to store numerical representations of documents and perform similarity-based retrieval. In this practical, FAISS is used to create a vector index and retrieve the top-k documents related to a query.

3. Objectives

- Generate embeddings for a collection of documents.
- Create a FAISS vector index for storing the embeddings.
- Insert all document vectors into the FAISS index.
- Convert a user query into a vector representation.
- Perform similarity search using the FAISS index.
- Retrieve and rank the top-k most similar documents.

4. FAISS Architecture

FAISS Vector Database Retrieval Architecture

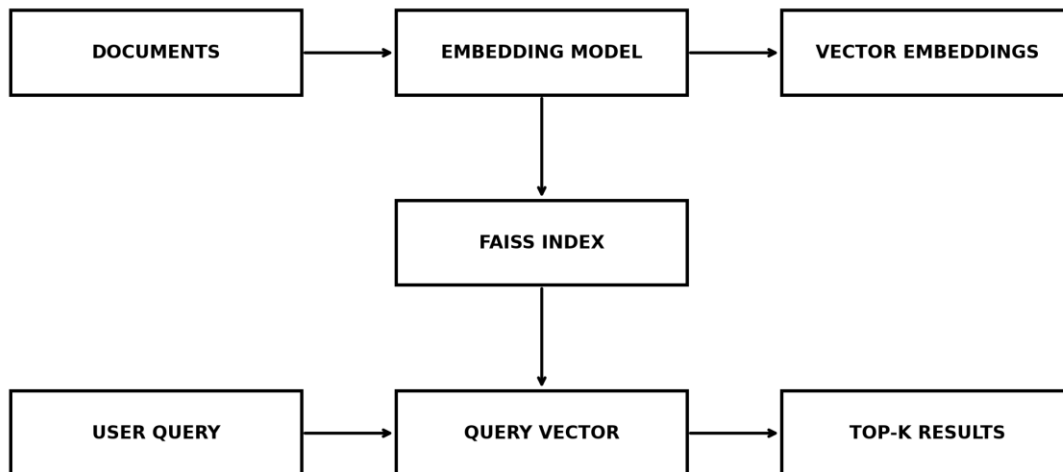


Figure 1: FAISS-based vector database architecture.

5. What is FAISS?

FAISS, or Facebook AI Similarity Search, is a library designed for efficient similarity search over dense vectors. It provides indexing structures that allow vector collections to be searched quickly. In this implementation, IndexFlatIP is used. The index performs inner-product search; after L2 normalization of the vectors, the inner product corresponds to cosine similarity.

6. Working Procedure

1. Load the Sentence Transformer model.
2. Define the document collection.
3. Generate 384-dimensional document embeddings.
4. Normalize the document vectors.
5. Create a FAISS IndexFlatIP index using the embedding dimension.
6. Add all document vectors to the FAISS index.
7. Generate an embedding for the user query and normalize it.
8. Request the top 5 nearest vectors from FAISS.
9. Map returned vector indices back to their original documents.
10. Display the ranked documents and database statistics.

7. FAISS Index Configuration

Parameter	Actual Value
Index Type	IndexFlatIP
Vector Dimension	384
Stored Vectors	8
Top-K Retrieved	5
Embedding Model	all-MiniLM-L6-v2
Search Type	Similarity-based vector retrieval

8. Installation

The FAISS package was installed using:

```
pip install faiss-cpu
```

The embedding and machine-learning libraries used by the implementation are:

```
pip install sentence-transformers scikit-learn
```

9. Python Implementation

```
from sentence_transformers import SentenceTransformer
import faiss
import numpy as np

model = SentenceTransformer("all-MiniLM-L6-v2")

documents = [
    "Machine learning enables computers to learn patterns from data.",
    "Artificial intelligence allows machines to perform intelligent tasks.",
    "Deep learning uses neural networks for advanced data analysis.",
    "Natural language processing helps computers understand human language.",
    "Computer vision enables machines to understand images and videos.",
    "Cloud computing provides scalable computing resources over the internet.",
    "Cybersecurity protects computer systems and networks from attacks.",
    "Database management systems store and organize structured information."
]

document_embeddings = model.encode(
    documents, convert_to_numpy=True
)

faiss.normalize_L2(document_embeddings)

dimension = document_embeddings.shape[1]
index = faiss.IndexFlatIP(dimension)

index.add(document_embeddings)

query = input("Enter your search query: ")

query_embedding = model.encode(
    [query], convert_to_numpy=True
)
faiss.normalize_L2(query_embedding)

top_k = 5
scores, indices = index.search(query_embedding, top_k)

for rank, (score, idx) in enumerate(
    zip(scores[0], indices[0]), 1
):
    print(f"Rank {rank}")
    print(f"FAISS Similarity Score : {score:.4f}")
    print(f"Document          : {documents[idx]}")

print("Index Type      :", "IndexFlatIP")
print("Vector Dimension :", dimension)
print("Stored Vectors   :", index.ntotal)
print("Top-K Retrieved  :", top_k)
```

10. Test Query

How can computers learn from data?

11. Actual Implementation Output

```
FAISS VECTOR DATABASE SEARCH RESULTS

Rank 1
FAISS Similarity Score : 0.6549
Document               : Machine learning enables computers to learn patterns from data.

Rank 2
FAISS Similarity Score : 0.4677
Document               : Computer vision enables machines to understand images and videos.

Rank 3
FAISS Similarity Score : 0.4490
Document               : Natural language processing helps computers understand human language.

Rank 4
FAISS Similarity Score : 0.4482
Document               : Artificial intelligence allows machines to perform intelligent tasks.

Rank 5
FAISS Similarity Score : 0.3676
Document               : Deep learning uses neural networks for advanced data analysis.

FAISS DATABASE INFORMATION
Index Type      : IndexFlatIP
Vector Dimension : 384
Stored Vectors  : 8
Top-K Retrieved : 5

TOP RETRIEVED DOCUMENT
Machine learning enables computers to learn patterns from data.
Score: 0.6549
```

Figure 2: Actual FAISS search output obtained during implementation.

12. Top-K Retrieval Results

Rank	Score	Retrieved Document	Position
1	0.6549	Machine learning enables computers to learn patterns from data.	Top-1
2	0.4677	Computer vision enables machines to understand images and videos.	Top-2
3	0.4490	Natural language processing helps computers understand human language.	Top-3
4	0.4482	Artificial intelligence allows machines to perform intelligent tasks.	Top-4
5	0.3676	Deep learning uses neural networks for advanced data analysis.	Top-5

13. Retrieval Performance Visualization

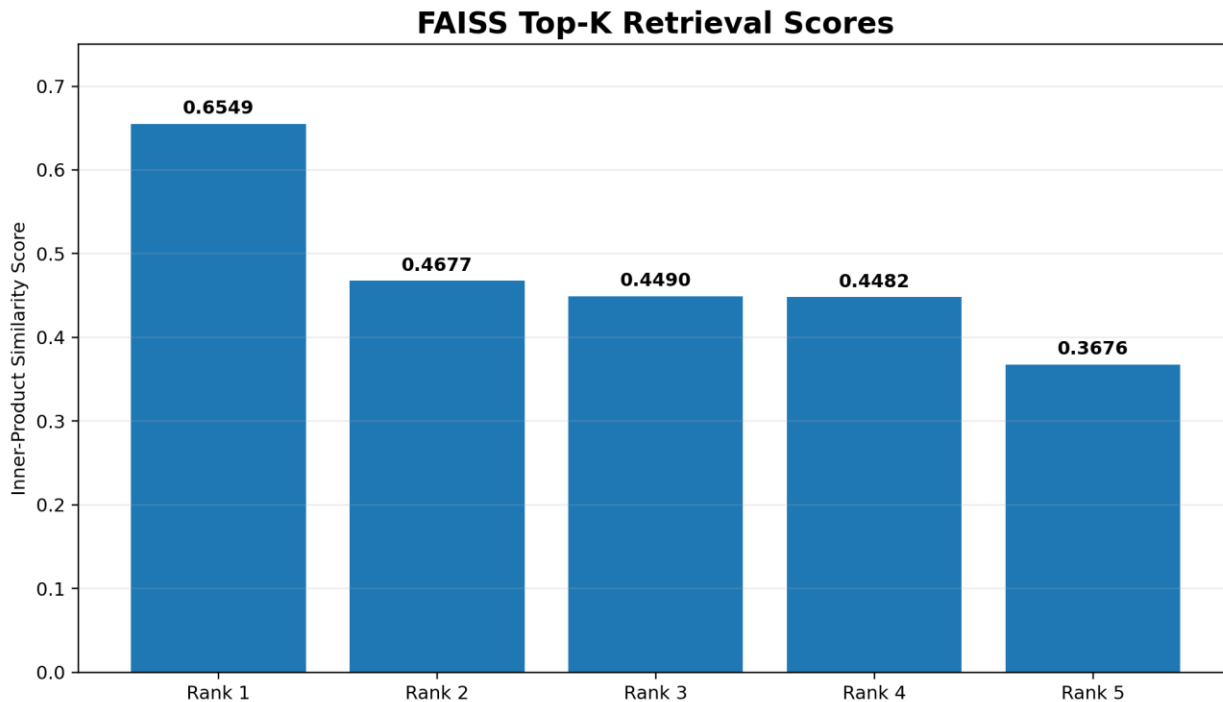


Figure 3: Top-K similarity scores returned by the FAISS vector index.

14. Result Analysis

The FAISS vector database successfully stored 8 document vectors, each with 384 dimensions. For the query “How can computers learn from data?”, FAISS returned five nearest documents. The machine learning document achieved the highest similarity score of 0.6549, followed by computer vision at 0.4677 and natural language processing at 0.4490. The system therefore correctly placed the machine learning document at Rank 1.

15. Advantages

- Efficient similarity search over dense vector representations.
- Supports large-scale vector indexing and retrieval.
- Provides direct Top-K nearest-neighbor search.
- Separates document storage/indexing from the query process.
- Can serve as a retrieval layer for RAG applications.

16. Applications

- Vector-based document retrieval
- Semantic search engines
- Recommendation systems
- Image and multimedia similarity search
- RAG retrieval systems
- Knowledge-base search

17. Conclusion

A FAISS-based vector database was successfully built and tested. Eight document embeddings were generated and stored in an IndexFlatIP vector index. A user query was converted into an embedding and used to retrieve the top five similar documents. The highest-ranked document was the machine learning document with a score of 0.6549. This implementation demonstrates the use of FAISS as a practical vector retrieval layer for modern semantic search and RAG systems.